

Mini-project 2 in Advanced Machine Learning (02460)

by Jacob Nielsen (s204093), Botong Yu (s214623), Malthe Bresler (s214631) & Carl Nørlund (s214624) on 2 April 2025

Part A: pullback geodesics

We implemented geodesics under the pull-back metric for a VAE trained on MNIST limited to the first 3 classes. We used the hand-out code as a starting point, and trained the VAE with a latent dimension of 2, a learning rate of 0.001, a batch size of 128 and running 400 epochs. These hyper-parameters were selected based on trial and error. We used squared energy minimization via torch to find the geodesics of the latent space. Below is a plot of the latent variables, as well as geodesics between 25 pairs.

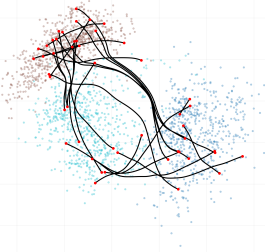


Figure 1: Geodesics between randomly sampled latent variables. Classes: dark blue is 0, red is 1 and light blue is 2.

We see clearly, in the plots, how the geodesics follow the data, taking paths through denser regions and avoiding regions with large uncertainty (fewer data-points). We can also see 'information highways', where multiple geodesics end up roughly following the same path. For example, moving from class 0 to 1 - which are further from each other in the latent space, the geodesics find a path through a region that contains a few 0's and 1's spread further from the cluster centers. Clearly, taking the path through 2 territory is slower. Finally, we noted that the geodesics are reliable across repeats for the same latent space, but if the encoder changes, the geodesics change with it - but they do seem to follow the same rough patterns; avoiding dead space and going through familiar regions.

Part B: Ensemble VAE geometry

We trained a series of Variational Autoencoders (VAEs) using a single encoder paired with an ensemble of K independent decoders, with K ranging from $K \in \{1, \dots, 9\}$. Each model decoder was trained for 200 epochs with a batch size of 256.

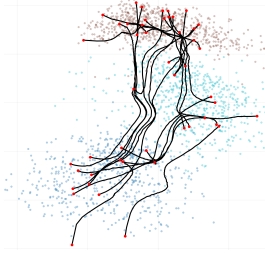


Figure 2: Plot of the same geodesics between latent variables as in Plot 1 calculated using 3 decoders.

Qualitatively the plotted geodesics computed using 3 decoders as seen in Figure 2 look very similar to the ones in Figure 1. However, the paths do seem to curve more aggressively which might be attributed to the ensemble metric penalizing regions where decoders disagree. In low-density regions the decoders are less constrained by data and thus produce more varied outputs, making paths through these regions more expensive under the ensemble energy.

To quantify the reliability of the learned geometry, we computed the Coefficient of Variation (CoV) of both geodesic and euclidean distances across 20 reruns for each ensemble size, evaluated on 15 fixed test point pairs.

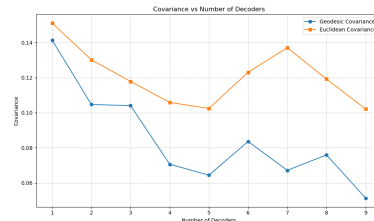


Figure 3: The coefficient of variation across multiple training runs for different number of decoders.

In Figure 3 we see the CoV of both euclidean and geodesic distances as a function of the number of decoders. The euclidean CoV remains roughly flat across ensemble sizes since it is measured directly in latent space and does not depend on the decoders. The fluctuations we observe are due to differences in the encoder across runs. The geodesic CoV shows a general downward trend as the ensemble grows. This aligns with the expectation that adding decoders reduces the influence of arbitrary model weights on the geometry in sparse regions. That said, the trend is not perfect — for example, the CoV increases at $K = 6$ before dropping again — which likely reflects noise from the finite number of reruns and point pairs. Despite this, for larger ensembles the geodesic CoV tends to fall below the euclidean CoV suggesting that the geodesic distance becomes a more reliable measure when model arbitrariness in undersampled regions is mitigated through ensembling.

Code Snippets

Link to github repo: <https://github.com/carllGN/02460Miniprojects>

Pull-back curve energy

```
1 def standard_curve_energy(curve, decoder):
2     decoder = decoder[0]
3     mu = decoder(curve).mean
4     energy = torch.sum((mu[:-1] - mu[1:]) ** 2)
5
6     return energy
```

Discrete geodesic computation via energy minimization

```
1 def compute_geodesic(z_start, z_end, decoders, n_points=100, epochs=2000, lr=0.001):
2     t = torch.linspace(0, 1, n_points, device=z_start.device).unsqueeze(1)
3     initial_curve = z_start.unsqueeze(0) * (1 - t) + z_end.unsqueeze(0) * t
4     interior_points = nn.Parameter(initial_curve[1:-1].detach().clone())
5     optimizer = optim.Adam([interior_points], lr=lr)
6     for epoch in range(epochs):
7         optimizer.zero_grad()
8         curve = torch.cat([z_start.unsqueeze(0), interior_points, z_end.unsqueeze(0)], dim=0)
9         energy = curve_energy(curve, decoders)
10        energy.backward()
11        optimizer.step()
12    final_curve = torch.cat([z_start.unsqueeze(0), interior_points, z_end.unsqueeze(0)], dim=0)
13    return final_curve.detach()
```

Model-average curve energy with ensemble decoders

```
1 for i in range(N - 1):
2     # sample two random decoders l and k
3     l = torch.randint(0, n_decoders, (1,)).item()
4     k = torch.randint(0, n_decoders, (1,)).item()
5
6     zi = curve[i]
7     znext = curve[i+1]
8
9     fl_zi = decoders[l](zi.unsqueeze(0)).mean
10    fk_znext = decoders[k](znext.unsqueeze(0)).mean
11
12    energy += torch.sum((fl_zi - fk_znext) ** 2)
```

Monte Carlo estimation of geodesic distance

```
1 with torch.no_grad():
2     ....
3     for i in range(final_path.shape[0] - 1):
4         ....
5         for _ in range(n_samples):
6             l = torch.randint(0, len(model.decoders), (1,)).item()
7             k = torch.randint(0, len(model.decoders), (1,)).item()
8             fl = model.decoders[l](zcurr).mean
9             fk = model.decoders[k](znext).mean
10            seg_dist += torch.norm(fl - fk)
11
12            dist += seg_dist / n_samples
13    all_geodistances[pair_idx, run_idx] = dist
```

Declaration of use of generative AI

This declaration **must** be filled out and included as the 3rd page of the document. The questions apply to all parts of the work, including research, project writing, and coding.

- I/we have used generative AI tools: yes

If you answered *yes*, please complete the following sections. List the generative AI tools you have used:

- Google Gemini
- Claude

Describe how the tools were used:

What did you use the tool(s) for? We used the tools for understanding, plotting, debugging, proof-reading and coding.

At what stage(s) of the process did you use the tool(s)? Through the entire process.

How did you use or incorporate the generated output? Mostly to deepen our understanding and to work out solutions. Individual times, we've used code directly, but it is noted at the script.